MANUAL TECNICO

Brenda Elizabeth Robles Concepcion
100067220

SEMINARIO DE PROYECTOS II
Henry Marcelino Candelario Rosario

ARQUITECTURA TÉCNICA

La aplicación utiliza una arquitectura web cliente-servidor sin un backend propio. La lógica de interfaz y procesamiento se ejecuta en el navegador, mientras que Supabase proporciona autenticación, acceso a datos y funciones remotas sobre PostgreSQL.

Capa	Responsabilidad
Navegador / Frontend	HTML, CSS y JavaScript. Gestiona la interfaz, lectura de Excel, filtros, gráficos, agenda y exportación a PDF.
Servicios de Supabase	Autenticación, API de datos, políticas RLS y llamadas RPC utilizadas por la aplicación.
PostgreSQL	Persistencia de tickets, agenda, historial de soluciones y datos auxiliares del sistema.

TECNOLOGÍAS Y LIBRERÍAS

Tecnología	Uso en GESTIONA!
HTML5	Estructura de las pantallas.
CSS3 + Tailwind CSS CDN	Diseño visual, distribución y adaptación de la interfaz.
JavaScript	Lógica del sistema y comunicación con Supabase.
Supabase JS v2	Autenticación, consultas y funciones RPC.
PostgreSQL / Supabase	Persistencia de la información.
SheetJS 0.18.5	Lectura de archivos .xlsx y .xls.
Chart.js	Gráficos e indicadores del dashboard.

Tecnología	Uso en GESTIONA!
html2pdf.js	Generación de reportes y agenda en PDF.
EmailJS	Envío por correo de información del dashboard cuando se configura el servicio.

ESTRUCTURA DEL PROYECTO

HTML	Agenda
	Archivo
	Bienvenida
	Dashboard
	Index
	supabase
CSS	Styles
JS	Agenda
	Archivo
	Bienvenida
	Dasboard
	Login
	supabase

Archivo / módulo	Responsabilidad principal
index.html / login.js	Registro, inicio de sesión y validaciones de acceso.
bienvenida.html / bienvenida.js	Pantalla inicial y resumen de información disponible.
archivo.html / archivo.js	Carga, lectura, mapeo y envío de tickets desde Excel.
dashboard.html / dashboard.js	Indicadores, gráficos, filtros, tabla, PDF y correo.
agenda.html / agenda.js	Motor de prioridad, agenda, bloqueos, seguimiento y tickets similares.
JS/supabase.js	Conexión, sesión, navegación protegida y utilidades compartidas.
CSS/styles.css	Estilos generales y componentes visuales propios.

FUNCIONAMIENTO DE LOS MÓDULOS

Autenticación y sesión

El registro e inicio de sesión utilizan Supabase Auth. login.js valida correo, contraseña y confirmación de contraseña antes de enviar la solicitud. Las pantallas internas ejecutan protegerPagina(), que verifica la sesión y redirige al login cuando el usuario no está autenticado.

Los mensajes de autenticación se traducen a mensajes comprensibles en español.

El nombre mostrado en la navegación se obtiene de user_metadata.nombre o del correo del usuario.

El cierre de sesión se realiza con db.auth.signOut().

Carga e importación de tickets

archivo.js acepta archivos .xlsx y .xls, lee la primera hoja mediante SheetJS y detecta automáticamente las columnas principales: ticket, estado, resumen, prioridad y fecha. El usuario puede corregir el mapeo antes de guardar.

El número de ticket es obligatorio; las filas sin identificador se omiten.

Si un ticket se repite dentro del mismo Excel, se conserva la última aparición.

Las fechas se normalizan al formato ISO aaaa-mm-dd cuando es posible.

La fila completa del Excel se conserva para permitir dashboards con columnas variables.

Los registros se envían en lotes de 250 a la función RPC subir_tickets; posteriormente se consulta el historial de cargas.

Dashboard

El dashboard consulta los tickets persistidos en Supabase y reconstruye dinámicamente las columnas a partir de los datos del Excel guardados. Permite aplicar filtros por rango de fechas, situación y columnas del archivo, además de generar KPIs, gráficos y una tabla de detalle.

Chart.js genera las visualizaciones de estado, prioridad y otras columnas categóricas detectadas.

La tabla limita la representación visual a 500 filas para evitar sobrecargar el navegador.

html2pdf.js genera una copia PDF del dashboard con los filtros aplicados.

EmailJS puede utilizarse para enviar un resumen cuando las credenciales del servicio están configuradas.

Agenda diaria y motor de prioridad

La agenda se construye a partir de los tickets pendientes del usuario. No incluye tickets cerrados en Jira ni tickets marcados como hechos. Los tickets bloqueados vuelven a ser elegibles cuando llega la fecha establecida para revisarlos.

Criterio	Regla utilizada
Prioridad del ticket	Crítica/Highest +50; Alta +35; Media/Normal +20; Baja +5.
Antigüedad	Suma hasta 60 puntos según los días abiertos.

Criterio	Regla utilizada
Arrastre de agenda	+5 por cada día anterior en agenda, hasta 25.
Estado Jira	Abierto/Pendiente +10; En proceso +5.
Bloqueo vencido	+15 cuando corresponde revisarlo nuevamente.

La agenda utiliza 10 tickets por día de forma predeterminada, aunque el usuario puede modificar el límite hasta un máximo de 50. Los tickets con mayor puntuación se colocan primero y los no atendidos permanecen disponibles para días posteriores.

Seguimiento, bloqueo y conocimiento técnico

Desde la agenda el usuario puede mantener un ticket pendiente, marcarlo como bloqueado o indicar que fue hecho. Al bloquearlo se registra de quién depende y la fecha en la que debe volver a mostrarse. Al resolverlo se puede documentar la solución aplicada.

La aplicación consulta la función RPC tickets_parecidos para mostrar incidencias similares y, cuando existe información previa, presentar al usuario el ticket relacionado, porcentaje de similitud y solución disponible. Esta función convierte las resoluciones anteriores en una referencia para casos futuros.

EXPLICACIÓN DE FRAGMENTO DE CÓDIGO

Conexión

```
const SUPABASE_URL = 'https://cuykabjeeztscywwqcqh.supabase.co';
const SUPABASE_KEY = 'sb_publishable_RuiVxXqBMAIzf4zYhZ7yTg_O7cnmn-7';
const db = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

Datos y conexión. La URL es la dirección del proyecto en Supabase y la llave pública permite conectarse a él. createClient crea la conexión y la guarda en db; con db se habla con la base de datos desde todo el sistema.

```
async function obtenerUsuario() {
  const { data } = await db.auth.getSession();
  const sesion = data && data.session;
  if (!sesion) return null;

  const u = sesion.user;
  const meta = u.user_metadata || {};
  return {
    id: u.id,
    email: u.email,
    nombre: meta.nombre || (u.email || "").split("@")[0]
  };
}
```

Revisa si hay alguien con sesión iniciada. Pide la sesión a Supabase y, si existe, devuelve el id, el correo y el nombre del usuario. Si no hay sesión devuelve null.

```
async function cerrarSesion() {
  await db.auth.signOut();
  location.href = "login.html";
}
```

Cierra la sesión del usuario (signOut) y lo regresa a la pantalla de login.

Lectura Del Excel Y Detección Automática De Columnas

```
/* ============================================================ */
function procesarArchivo(archivo) {

  if (!archivo) return;

  const tiposPermitidos = [
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "application/vnd.ms-excel"
  ];

  // Mejora seminario de proyectos II BR: también se acepta por la extensión del nombre
  // (algunos navegadores en Windows no informan el tipo del archivo)
  const esExcel = tiposPermitidos.includes(archivo.type) || /\.xlsx?$/i.test(archivo.name);

  if (!esExcel) {
    alert("Solo se permiten archivos Excel (.xlsx o .xls)");
    inputArchivo.value = "";
    return;
  }
```

La función procesarArchivo revisa que el archivo que la usuaria eligió sea un Excel antes de leerlo. Primero, si no llegó ningún archivo (if (!archivo) return;), la función termina sin hacer nada. Luego define la lista tiposPermitidos, con los dos tipos que el navegador usa para identificar un Excel: .xlsx, el formato moderno, y .xls, el antiguo. En la mejora del Seminario II se agregó que también se acepte por la extensión del nombre, porque algunos navegadores en Windows no informan el tipo del archivo. Por eso la constante esExcel queda en verdadero si el tipo está en la lista o si el nombre termina en .xls o .xlsx; esto último lo comprueba la expresión regular /\.xlsx?$/i, que ignora mayúsculas. Si el archivo no es Excel, el programa muestra la alerta “Solo se permiten archivos Excel (.xlsx o .xls)”, vacía el campo de archivo (inputArchivo.value = "") para que se pueda volver a elegir el mismo archivo u otro, y termina con return, de modo que nunca se procesa un archivo inválido.

Construcción de registros y control de duplicados

```
function construirRegistros() {
  const mapa = leerMapa();
  const unicos = new Map();
  let sinNumero = 0;
  let repetidosEnArchivo = 0;

  filasExcel.forEach(fila => {
    const id = textoDe(fila, mapa.ticket);
    if (!id) { sinNumero++; return; }
    if (unicos.has(id)) repetidosEnArchivo++;

    unicos.set(id, {
      identificador_tk: id,
      estado: textoDe(fila, mapa.estado),
      resumen: textoDe(fila, mapa.resumen),
      prioridad: textoDe(fila, mapa.prioridad),
      fecha_creacion: mapa.fecha ? aISO(fila[mapa.fecha]) : null,
      datos: limpiarFila(fila)
    });
  });
```

construirRegistros convierte las filas del Excel en la lista de tickets que se enviará a la base de datos. Recorre cada fila y toma el número de ticket; si no tiene número la descarta, y si el número ya apareció, no lo duplica (usa un Map) y lo cuenta como repetido. Con cada ticket válido arma un registro con el número, estado, resumen, prioridad y fecha de creación (convertida con aISO), y guarda además la fila completa en datos para no perder las demás columnas. Al final quedan los tickets únicos listos para guardar y los contadores de filas sin número y repetidas.

Agenda



```
46  function calcularScore(t, veces) {
47    let score = 0;
48
49    const prioridad = String(t.prioridad || "");
50    if (/highest|critic|cr[ií]tic|urgent|blocker|bloqueante|m[aá]s alta|muy alta/i.test(prioridad)) score +
51    else if (/high|alta|major/i.test(prioridad)) score += 35;
52    else if (/medium|media|normal/i.test(prioridad)) score += 20;
53    else if (/low|baja|minor|trivial/i.test(prioridad)) score += 5;
54
55    if (t.fecha_creacion) score += Math.min(Math.max(0, diasEntre(t.fecha_creacion, hoyISO())), 60);
56
57    score += Math.min((veces || 0) * 5, 25);
58
59    if (/abiert|open|nuev|pendiente|por hacer|to do/i.test(t.estado)) score += 10;
60    else if (/proceso|progreso|curso|progress/i.test(t.estado)) score += 5;
61
62    if (t.situacion === "bloqueado") score += 15;   // ya venci5 su bloqueo: toca revisarlo
63
64    return score;
65  }
66
```

calcularScore le asigna a cada ticket un puntaje de prioridad para decidir en qué orden aparece en la agenda: entre más puntos, más arriba. Empieza en 0 y suma así:

- Prioridad (líneas 49 a 53): busca palabras en el texto de la prioridad. Crítica o highest suma 50, alta o high suma 35, media o normal suma 20 y baja o low suma 5. Reconoce español e inglés, y las else if aseguran que solo cuente una.
- Antigüedad (línea 55): suma un punto por cada día desde que se creó el ticket, hasta un máximo de 60. Un ticket viejo sube en la lista.
- Arrastre (línea 57): veces es cuántos días ha pasado el ticket sin hacerse. Suma 5 por cada día, hasta un máximo de 25.
- Estado (líneas 59 y 60): abierto, nuevo o pendiente suma 10; en proceso suma 5.
- Bloqueo (línea 62): si estaba bloqueado y ya venció su plazo, suma 15 para que se revise.

Al final return score devuelve el total.



```
try {
  for (let i = 0; i < registros.length; i += LOTE) {
    boton.textContent = `Guardando ${Math.min(i + LOTE, registros.length)} de ${registros.length}...`;

    const { data, error } = await db.rpc("subir_tickets", {
      p_tickets: registros.slice(i, i + LOTE),
      p_nombre_archivo: archivoTemporal.name,
      p_carga_id: cargaId
    });
    if (error) throw error;

    cargaId = data.carga_id;
    total.nuevos += data.nuevos;
    total.actualizados += data.actualizados;
    total.ajenos += data.ajenos;
    total.lista.push(...(data.ajenos_lista || []));
```

Este bloque de código envía los tickets a la base de datos por partes (lotes) y va sumando el resultado de cada envío. Dentro de un try, el ciclo for avanza de LOTE en LOTE sobre la lista registros, y en cada vuelta cambia el texto del botón para mostrar el avance (“Guardando 250 de 1000...”; Math.min evita pasarse del total en el último lote). Luego llama a la función de la base de datos subir_tickets con db.rpc, enviándole el pedazo de tickets (registros.slice(i, i + LOTE)), el nombre del archivo y el número de carga. Si Supabase responde con error, throw error detiene todo y salta al catch. Si todo sale bien, guarda el carga_id que devolvió la base (para que todos los lotes cuenten como una sola carga) y suma a total los tickets nuevos, los actualizados, los que ya eran de otra persona (ajenos) y la lista de esos números, para mostrar el resumen al final.



```
  score += Math.min((veces || 0) * 5, 25);

  if (/abiert|open|nuev|pendiente|por hacer|to do/i.test(t.estado)) score += 10;
  else if (/proceso|progreso|curso|progress/i.test(t.estado)) score += 5;

  if (t.situacion === "bloqueado") score += 15;   // ya venció su bloqueo: toca revisarlo

  return score;
}

/* ============================================================
   GENERAR LA AGENDA DEL DÍA
============================================================ */

async function generarAgenda() {
  if (fechaActual < hoyISO()) {
    setEstado("Solo puedes generar la agenda de hoy o de días que vienen. Los días pasados se ven como hi
    return;
  }

  const boton = $("btnGenerar");
  boton.disabled = true;
```

Este código termina de calcular la prioridad de un ticket (su “score”) y empieza la función que genera la agenda del día. Al score se le suman puntos por los días que el ticket lleva pasando de un día a otro (5 por día, máximo 25), 10 puntos si el estado es abierto, nuevo o pendiente, o 5 si está en proceso, y 15 puntos más si estaba bloqueado y ya venció su bloqueo, para que se revise. Al final return score devuelve el total; entre más alto, más arriba queda el ticket en la agenda.

Después empieza generarAgenda. Primero valida la fecha: si el día elegido es anterior a hoy (fechaActual < hoyISO()), muestra el aviso de que solo se puede generar la agenda de hoy o de días futuros y se sale con return. Si la fecha es válida, toma el botón “Generar” y lo desactiva (disabled = true) para evitar que se presione dos veces mientras trabaja

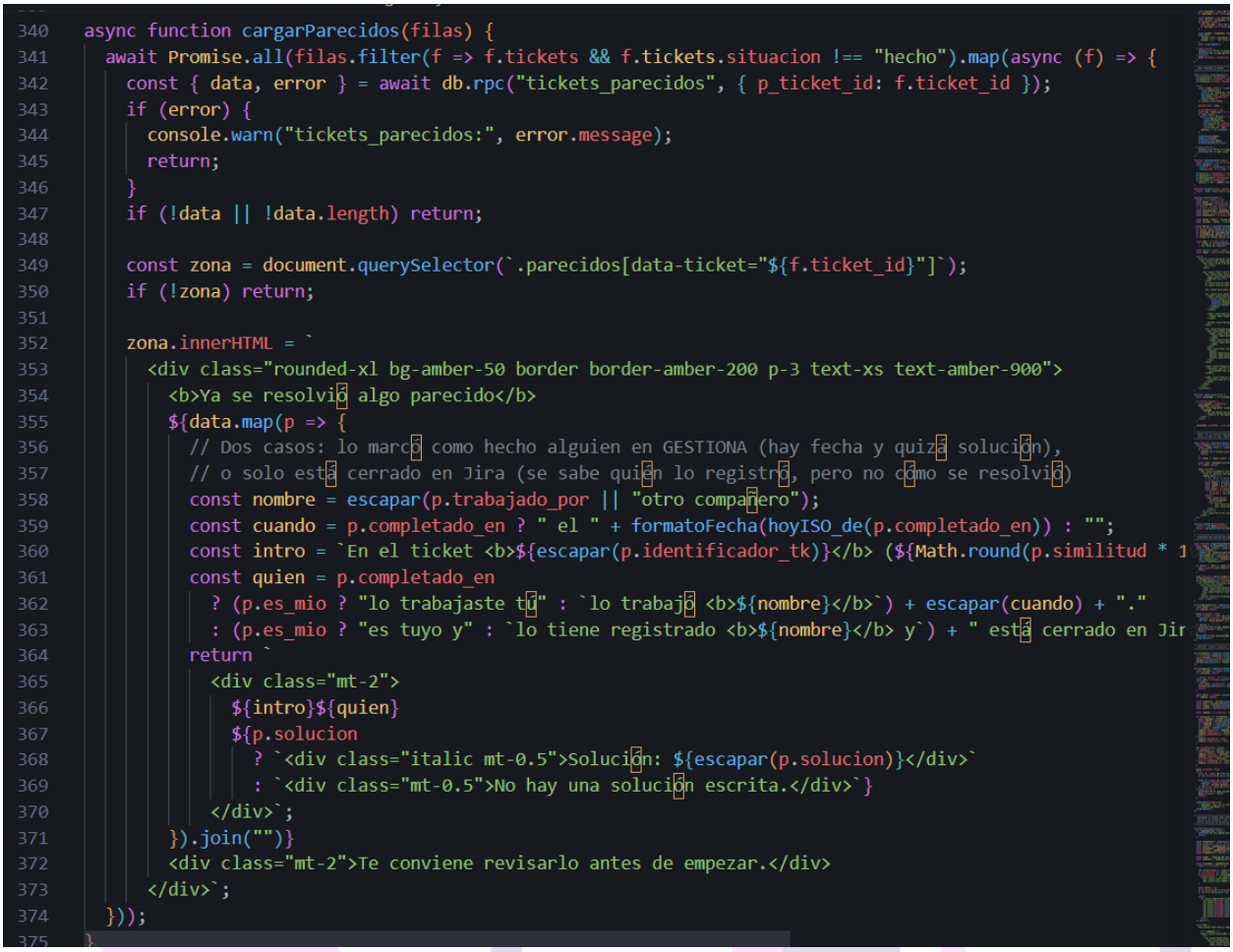


```
async function cargarParecidos(filas) {
  await Promise.all(filas.filter(f => f.tickets && f.tickets.situacion !== "hecho").map(async (f) => {
    const { data, error } = await db.rpc("tickets_parecidos", { p_ticket_id: f.ticket_id });
    if (error) {
      console.warn("tickets_parecidos:", error.message);
      return;
    }
    if (!data || !data.length) return;

    const zona = document.querySelector(`.parecidos[data-ticket="${f.ticket_id}"]`);
    if (!zona) return;

    zona.innerHTML = `
      <div class="rounded-xl bg-amber-50 border border-amber-200 p-3 text-xs text-amber-900">
        <b>Ya se resolvió algo parecido</b>
        ${data.map(p => {
          // Dos casos: lo marcó como hecho alguien en GESTIONA (hay fecha y quizá solución),
          // o solo está cerrado en Jira (se sabe quién lo registró, pero no cómo se resolvió)
          const nombre = escapar(p.trabajado_por || "otro compañero");
          const cuando = p.completado_en ? " el " + formatoFecha(hoyISO_de(p.completado_en)) : "";
          const intro = `En el ticket <b>${escapar(p.identificador_tk)}</b> (${Math.round(p.similitud * 1
          const quien = p.completado_en
            ? (p.es_mio ? "lo trabajaste tú" : `lo trabajó <b>${nombre}</b>`) + escapar(cuando) + "."
            : (p.es_mio ? "es tuyo y" : `lo tiene registrado <b>${nombre}</b> y`) + " está cerrado en Jir
          return `
            <div class="mt-2">
              ${intro}${quien}
              ${p.solucion
                ? `<div class="italic mt-0.5">Solución: ${escapar(p.solucion)}</div>`
                : `<div class="mt-0.5">No hay una solución escrita.</div>`}
            </div>`;
        }).join("")}
        <div class="mt-2">Te conviene revisarlo antes de empezar.</div>
```

cargarParecidos revisa, para cada ticket de la agenda que aún no está hecho, si ya se resolvió un caso parecido, y si lo hay muestra un aviso amarillo dentro de la tarjeta del ticket. Recorre los tickets pendientes (filter descarta los que ya están en “hecho”) y, para cada uno, consulta a la base de datos con db.rpc("tickets_parecidos"). Si hay error, lo deja en la consola y sigue; si no hay parecidos, no muestra nada. Cuando sí los hay, busca en la página el espacio reservado de ese ticket (.parecidos[data-ticket=...]) y le escribe el aviso con innerHTML: “Ya se resolvió algo parecido”, con el ticket similar y el porcentaje de parecido. Después indica quién lo resolvió y cuándo: “lo trabajaste tú”, “lo trabajó Juan el 14/10/2026” o, si solo está cerrado en Jira, “lo tiene registrado Juan y está cerrado en Jira”. Añade la solución escrita, o “No hay una solución escrita” si no existe, y cierra con “Te conviene revisarlo antes de empezar”. escapar protege el texto para que no rompa el HTML.



```
async function cargarParecidos(filas) {
  await Promise.all(filas.filter(f => f.tickets && f.tickets.situacion !== "hecho").map(async (f) => {
    const { data, error } = await db.rpc("tickets_parecidos", { p_ticket_id: f.ticket_id });
    if (error) {
      console.warn("tickets_parecidos:", error.message);
      return;
    }
    if (!data || !data.length) return;

    const zona = document.querySelector(`.parecidos[data-ticket="${f.ticket_id}"]`);
    if (!zona) return;

    zona.innerHTML = `
      <div class="rounded-xl bg-amber-50 border border-amber-200 p-3 text-xs text-amber-900">
        <b>Ya se resolvió algo parecido</b>
        ${data.map(p => {
          // Dos casos: lo marcó como hecho alguien en GESTIONA (hay fecha y quizá solución),
          // o solo está cerrado en Jira (se sabe quién lo registró, pero no cómo se resolvió)
          const nombre = escapar(p.trabajado_por || "otro compañero");
          const cuando = p.completado_en ? " el " + formatoFecha(hoyISO_de(p.completado_en)) : "";
          const intro = `En el ticket <b>${escapar(p.identificador_tk)}</b> (${Math.round(p.similitud * 1
          const quien = p.completado_en
            ? (p.es_mio ? "lo trabajaste tú" : `lo trabajó <b>${nombre}</b>`) + escapar(cuando) + "."
            : (p.es_mio ? "es tuyo y" : `lo tiene registrado <b>${nombre}</b> y`) + " está cerrado en Jir
          return `
            <div class="mt-2">
              ${intro}${quien}
              ${p.solucion
                ? `<div class="italic mt-0.5">Solución: ${escapar(p.solucion)}</div>`
                : `<div class="mt-0.5">No hay una solución escrita.</div>`}
            </div>`;
        }).join("")}
        <div class="mt-2">Te conviene revisarlo antes de empezar.</div>
      </div>`;
  }));
}
```

Sirve para avisar, dentro de la tarjeta de cada ticket pendiente, si ya se resolvió un caso parecido.

Para cada ticket que no está “hecho”, consulta a la base de datos con tickets_parecidos. Si hay error lo deja en la consola, y si no hay parecidos no muestra nada. Si los hay, busca el espacio de ese ticket en la página y escribe en él un recuadro amarillo con el título “Ya se resolvió algo parecido”. Por cada caso parecido indica:

- **El ticket y su porcentaje de parecido.**
- **Quién lo resolvió y cuándo.** Puede decir “lo trabajaste tú”, “lo trabajó Juan el 14/10/2026” o, si solo está cerrado en Jira, “lo tiene registrado Juan y está cerrado en Jira”.
- **La solución**, o “No hay una solución escrita” si no se registró.

Cierra con “Te conviene revisarlo antes de empezar”. escapar protege los textos para que no rompan el HTML.

Dashboard



```
const filas = await traerTodos(() =>
  db.from("tickets")
    .select("identificador_tk, estado, situacion, fecha_creacion, datos")
    .order("id"));

if (!filas.length) {
  mostrarVacio();
  return;
}

// Todas las columnas que aparezcan en cualquier ticket (cada Excel puede traer columnas distintas)
const nombres = new Set();
filas.forEach(f => Object.keys(f.datos || {}).forEach(k => nombres.add(k)));
columnas = [...nombres];

if (columnas.length) {
  datosOriginales = filas.map(f => {
    const fila = {};
    columnas.forEach(c => { fila[c] = (f.datos && f.datos[c] !== undefined) ? f.datos[c] : ""; });
    metaPorFila.set(fila, { fecha: f.fecha_creacion, situacion: f.situacion });
    return fila;
  });
} else {
  // Respaldo: si un ticket no trae el detalle del Excel, se usan los campos básicos
  columnas = ["Ticket", "Estado"];
  datosOriginales = filas.map(f => {
    const fila = { "Ticket": f.identificador_tk, "Estado": f.estado };
    metaPorFila.set(fila, { fecha: f.fecha_creacion, situacion: f.situacion });
    return fila;
  });
}
```

Trae de la base de datos los tickets del usuario y los deja con la forma de tabla que el dashboard necesita para mostrarlos, filtrarlos y graficarlos.

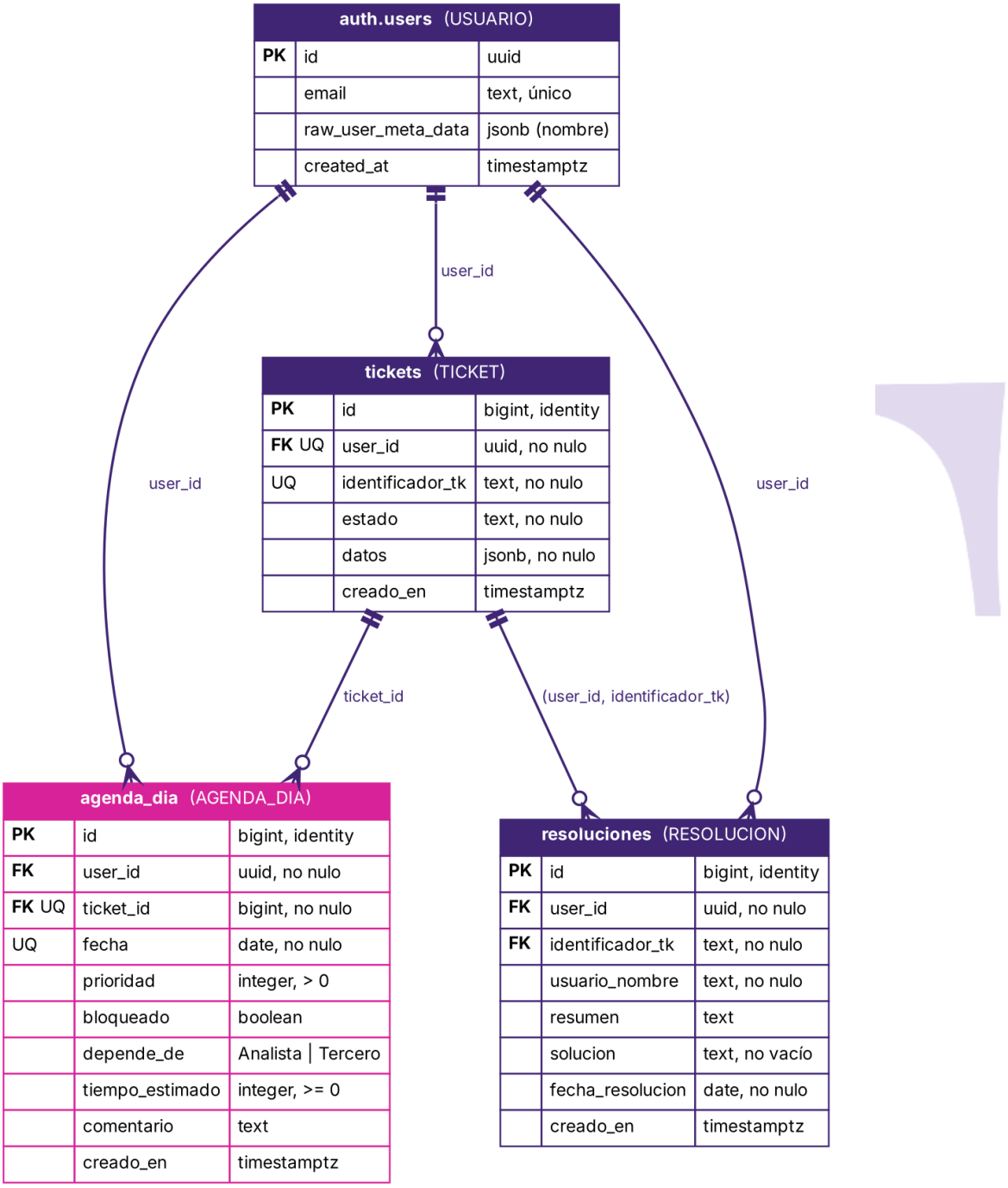
Primero traerTodos pide todos los tickets de la tabla (Supabase entrega máximo 1000 por vez, así que la función va pidiendo por partes hasta completarlos), con las columnas identificador, estado, situación, fecha y datos, que es la fila original del Excel. Si no hay ninguno (!filas.length), muestra el aviso de vacío con mostrarVacio() y termina.

Luego reúne los nombres de todas las columnas que aparecen en el campo datos de cualquier ticket, usando un Set para que no se repitan. Esto es necesario porque cada Excel puede traer columnas distintas.

Si encontró columnas, reconstruye cada ticket como una fila con todas esas columnas (si a un ticket le falta una, la deja vacía) y guarda aparte su fecha y situación en metaPorFila, para usarlas en los filtros. Si ningún ticket trae el detalle del Excel, usa como respaldo solo dos columnas, “Ticket” y “Estado”.

BASE DE DATOS Y SEGURIDAD

La persistencia se realiza en PostgreSQL mediante Supabase. Las entidades principales documentadas son usuarios, tickets, agenda diaria y resoluciones. La versión final del frontend también consulta información auxiliar de cargas para mostrar el historial de archivos procesados.



CONFIGURACIÓN Y DESPLIEGUE

Requisitos

Requisito	Uso
Navegador moderno	Chrome, Edge o equivalente con JavaScript habilitado.
Cuenta/proyecto Supabase	Autenticación y base de datos.
Servidor web local	Node.js con npx serve o servidor HTTP de Python.
Internet	Necesario para Supabase y las librerías cargadas por CDN.

Variables de conexión

La conexión principal se configura en SRC/JS/supabase.js. Al mover el proyecto a otra instancia, deben reemplazarse la URL y la clave pública sin colocar claves administrativas.

```
const SUPABASE_URL = 'https://TU-PROYECTO.supabase.co';
const SUPABASE_KEY = 'sb_publishable_...';
const db = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

Para el envío por correo, dashboard.js contiene la configuración de EmailJS. Si esa función se utilizará en otro ambiente, deben actualizarse PUBLIC_KEY, SERVICE_ID y TEMPLATE_ID con los valores correspondientes

Puesta en marcha

1. Crear o seleccionar el proyecto en Supabase y preparar la estructura de base de datos utilizada por la versión final.
2. Verificar autenticación, políticas RLS y funciones RPC requeridas por el frontend.
3. Configurar SUPABASE_URL y SUPABASE_KEY en SRC/JS/supabase.js.
4. Abrir una terminal en la carpeta del proyecto y publicar la carpeta SRC mediante un servidor web local.
5. Acceder desde el navegador, crear/iniciar sesión y validar una carga con los archivos de prueba incluidos en Data

MANTENIMIENTO Y SOLUCIÓN DE PROBLEMAS

Situación	Validación recomendada
No permite iniciar sesión	Validar credenciales, confirmación de correo, conexión a Supabase y configuración de Auth.
La página vuelve al login	Comprobar que exista una sesión activa y que JS/supabase.js cargue correctamente.
Excel no se procesa	Verificar formato .xlsx/.xls, primera fila con encabezados y que exista una columna de número de ticket.
Error "función no encontrada"	Verificar que las funciones RPC requeridas por el frontend estén creadas en Supabase.
Dashboard vacío	Confirmar que el usuario tenga tickets persistidos y que las políticas RLS permitan leerlos.
No aparecen tickets similares	Revisar la RPC tickets_parecidos y que existan tickets resueltos o cerrados comparables.
PDF no se genera	Revisar conexión a Internet y carga correcta de html2pdf.js.
Correo no se envía	Validar credenciales y plantilla configuradas en EmailJS.

CONSIDERACIONES PARA FUTURAS MODIFICACIONES

- Mantener separada la lógica compartida de sesión y utilidades en JS/supabase.js.
- Si cambia el formato de Excel, revisar los patrones de detección definidos en archivo.js antes de modificar la base de datos.
- Si se modifica el motor de prioridad, documentar la nueva ponderación y volver a ejecutar las pruebas de agenda.
- Después de cambiar políticas RLS, probar con al menos dos usuarios para confirmar el aislamiento de información.
- Conservar respaldo de la base y del código antes de cambios estructurales. El respaldo documentado actualmente es manual.
- Mantener las dependencias CDN actualizadas de manera controlada y volver a probar dashboard, Excel y PDF después de cualquier actualización.

CONCLUSIÓN TÉCNICA

GESTIONA! 2.0 está construido con una arquitectura web ligera en la que el navegador concentra la lógica de presentación y Supabase aporta autenticación y persistencia. La separación por módulos permite identificar con claridad las responsabilidades de acceso, carga de datos, dashboard y agenda. El motor de prioridad, el seguimiento de bloqueos y la consulta de tickets similares constituyen los componentes principales incorporados en esta versión.

Para mantener el sistema estable es importante conservar la correspondencia entre el frontend y la estructura de Supabase, proteger el acceso mediante RLS, evitar el uso de claves administrativas en el cliente y realizar pruebas posteriores a cualquier cambio en la base de datos, funciones RPC o lógica de priorización.